

Técnicas, Parâmetros e Resultados

Análise de Tabuleiros de Xadrez — Documentação Técnica

Davi Ludvig e Julia Macedo

Disciplina INE410121 / TRV410001 — Visão Computacional - UFSC

Dataset: Synthetic Chess Board Images (Kaggle, thefamousrat)

Visão Geral da Pipeline

O sistema implementa uma pipeline híbrida: técnicas clássicas de CV para detecção de tabuleiro, ocupação e cor; e deep learning (transfer learning) para identificação do tipo de peça.

```
Imagem bruta (1280×1280 JPEG)
├─ 1. Pré-processamento      → tons de cinza + blur gaussiano
├─ 2. Detecção de bordas      → Canny
├─ 3. Detecção de linhas      → Transformada de Hough
├─ 4. Correção de perspectiva → homografia (warp)
├─ 5. Segmentação 8×8        → 64 células de 60×60 px
├─ 6. Detecção de ocupação    → votação de features clássicas
├─ 6.5 Classificação de cor   → threshold HSV
├─ 7. Detecção de jogadas     → comparação temporal
└─ 9. Tipo de peça           → ResNet-34 transfer learning
```

1. Pré-processamento

Conversão para tons de cinza

A maioria dos operadores de borda opera sobre imagens monocromáticas. A conversão usa a luminância perceptual (padrão OpenCV `COLOR_BGR2GRAY`):

$$Y = 0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R$$

Suavização gaussiana

Filtragem no domínio espacial para reduzir ruído de alta frequência antes da detecção de bordas:

- **Kernel:** 5×5
- **σ :** calculado automaticamente pelo OpenCV ($\sigma = 0.3 \cdot ((\text{ksize}-1) \cdot 0.5 - 1) + 0.8$)
- **Função:** `cv2.GaussianBlur(gray, (5,5), 0)`

Por que suavizar antes? Detectores de borda respondem ao gradiente de intensidade; ruído de alta frequência gera gradientes espúrios que produzem bordas falsas.

Análise de histograma e equalização

Técnica	Função OpenCV	Efeito
Histograma	<code>cv2.calcHist</code>	Diagnóstico da distribuição de intensidade
Equalização global	<code>cv2.equalizeHist</code>	Redistribui intensidades para melhorar contraste global
CLAHE	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))</code>	Equalização local; preserva detalhes sem saturar regiões uniformes

A equalização não é aplicada diretamente na pipeline principal; serve como ferramenta de análise e diagnóstico do dataset.

2. Detecção de Bordas

Operadores comparados

Operador	Kernel	Características
Roberts	2×2 cruzado	Simples, muito sensível a ruído
Prewitt	3×3 média	Suavização uniforme
Sobel	3×3 ponderado	Mais peso no centro; padrão em muitos sistemas
Canny	Multi-etapa	Supressão de não-máximos + histerese; mais robusto

Canny — pipeline interna

1. **Suavização gaussiana** (já aplicada na etapa 1)
2. **Gradiente Sobel** — magnitude e direção em cada pixel
3. **Supressão de não-máximos** — afina bordas a 1 pixel de largura
4. **Histerese** — dois limiares determinam bordas fortes e fracas:
 - Pixel com gradiente > **high** → borda forte
 - Pixel entre **low** e **high** → borda fraca (aceito somente se conectado a uma borda forte)
 - Pixel < **low** → descartado

Parâmetros escolhidos: **low=50, high=150**

Configuração	Efeito
(30, 80) — sensível	Muitas bordas, inclusive ruído
(50, 150) — equilibrado	Bom compromisso; usado na pipeline
(80, 200) — conservador	Apenas bordas fortes

Operações morfológicas

Aplicadas sobre a imagem binária de bordas para limpeza e conexão:

Operação	Kernel	Efeito
Erosão	3×3	Remove pixels isolados (ruído pontual)
Dilatação	3×3	Expande bordas; conecta segmentos próximos
Abertura (erosão → dilatação)	5×5	Remove ruído sem alterar tamanho das bordas
Fechamento (dilatação → erosão)	5×5	Preenche lacunas nas bordas sem alterar tamanho

3. Detecção de Linhas — Transformada de Hough

Conceito

A Transformada de Hough converte o problema de detectar linhas no espaço da imagem em detectar picos no espaço paramétrico (ρ , θ):

- Cada pixel de borda (x , y) vota em todas as curvas (ρ , θ) do espaço acumulador
- Picos do acumulador acima do threshold indicam linhas reais
- $\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$

Função: `cv2.HoughLines(edges, rho=1, theta=π/180, threshold=100)`

Parâmetros

Parâmetro	Valor	Papel
<code>rho</code>	1 px	Resolução da distância
<code>theta</code>	π/180 rad	Resolução angular (1°)
<code>threshold</code>	80–100	Mínimo de votos para aceitar uma linha

Classificação e seleção das linhas

As linhas detectadas são classificadas em horizontais ($\theta \approx \pi/2$) e verticais ($\theta \approx 0$). Um algoritmo de janela deslizante (`_best_grid_window`) seleciona as 9 linhas mais igualmente espaçadas em cada direção, correspondendo às 9 linhas de grade do tabuleiro.

Detalhe importante: o dataset inclui uma borda de madeira ao redor do grid de jogo. A última linha horizontal detectada (`h_grid[-1]`) corresponde a essa borda e é descartada — os cantos são calculados usando `h_grid[-2]` como borda inferior real.

4. Correção de Perspectiva

Homografia

Uma homografia é uma transformação projetiva 2D (matriz 3×3) que mapeia 4 pontos de origem a 4 pontos de destino. Usamos os 4 cantos detectados do tabuleiro como pontos de origem e os cantos de

um quadrado `DST_SIZE×DST_SIZE` como destino.

```
H, _ = cv2.findHomography(src_pts, dst_pts)
warped = cv2.warpPerspective(img, H, (DST_SIZE, DST_SIZE))
```

Parâmetros:

- `DST_SIZE = 480` px — imagem retificada
- Cada célula resultante: $480/8 = 60 \times 60$ px

Detecção automática de cantos

A função `detect_board_corners_combined` executa a pipeline completa de Hough e retorna os 4 cantos detectados sem usar anotações. Em caso de falha, os cantos anotados no JSON do dataset são usados como fallback.

Compensação de rotação

Como a câmera fotografa o tabuleiro de ângulos variados, o mapa de ocupação pode estar rotacionado 0°, 90°, 180° ou 270° em relação ao ground truth. A função `best_rotation_vs_gt` testa as 4 rotações e usa a que maximiza a concordância com o GT.

5. Segmentação 8×8

Após a correção de perspectiva, o tabuleiro ocupa toda a imagem `480×480`. A divisão é feita por fatiamento uniforme:

```
cell_h, cell_w = DST_SIZE // 8, DST_SIZE // 8
cells[r][c] = warped[r*cell_h:(r+1)*cell_h, c*cell_w:(c+1)*cell_w]
```

Cada célula resultante tem **60×60 pixels** e corresponde a uma casa (A1–H8) do tabuleiro.

6. Detecção de Ocupação

Features clássicas

Para cada célula, extraímos 4 descritores:

Feature	Cálculo	Threshold	Lógica
Desvio-padrão de intensidade	<code>np.std(gray_cell)</code>	18	Peças criam variação de brilho
Densidade de bordas	<code>np.mean(canny_cell > 0)</code>	0.04	Peças geram bordas internas

Feature	Cálculo	Threshold	Lógica
Variância do Laplaciano	<code>cv2.Laplacian(cell).var()</code>	80	Indica textura/nitidez (peça ≠ madeira lisa)
Diferença centro-borda	<code>mean(centro) - mean(borda)</code>	12	Peça geralmente ocupa o centro da célula

Classificador por votação

```

votes = sum([
    std_intensity > 18,
    edge_density > 0.04,
    texture_var > 80,
    center_diff > 12,
])
occupied = votes >= 2 # limiar: maioria simples (≥2 de 4)

```

Justificativa do limiar ≥2: com material uniforme (madeira sobre madeira), nenhuma feature individual é confiável o suficiente. A combinação reduz falsos positivos e negativos.

Resultados (10 imagens, detecção automática de cantos)

Métrica	Média	Mediana	Mín	Máx
Acurácia	73.1%	64.8%	43.8%	100%
Precisão	71.8%	63.4%	38.6%	100%
Recall	87.5%	92.0%	48.4%	100%
F1	77.2%	72.5%	50.8%	100%

Observação: recall alto (87.5%) indica que a maioria das peças é detectada, mas a precisão menor revela falsos positivos — o detector tende a marcar células vazias como ocupadas devido ao baixo contraste do dataset.

Média de peças GT: 32.1 | **Média detectada:** 41.3 — o pipeline superestima a ocupação.

6.5 Classificação de Cor

A cor de cada peça (clara ou escura) é determinada pelo canal **V (Value)** no espaço HSV:

```

hsv = cv2.cvtColor(cell, cv2.COLOR_BGR2HSV)
mean_v = np.mean(hsv[:, :, 2])
color = "w" if mean_v > threshold else "b"

```

Lógica: peças claras (boxwood) têm maior brilho (V alto); peças escuras (ebony) têm V menor.

7. Detecção de Jogadas

A comparação temporal entre dois mapas de ocupação identifica casas que mudaram de estado:

```
def detect_moves(occ_a, occ_b):  
    emptied   = occ_a & ~occ_b    # estava ocupada, agora vazia  
    occupied  = ~occ_a & occ_b    # estava vazia, agora ocupada  
    return list of (row, col, change_type)
```

Para uma jogada simples, espera-se **1 casa esvaziada + 1 casa ocupada**. Capturas resultam em 1 esvaziada + 0 ou 1 ocupada (dependendo da implementação).

Na demonstração, usamos o ground truth de ocupação para isolar a avaliação do detector de mudanças da qualidade do classificador clássico.

9. Classificação de Tipo de Peça — ResNet-34

Motivação

A abordagem clássica (template matching, HOG, Hu Moments) mostrou-se insuficiente para distinguir as 12 classes de peças dado:

- Baixo contraste peça-tabuleiro (madeira uniforme)
- Deformação causada pela câmera em ângulo
- Silhuetas similares (peão × bispo × torre em perspectiva)

Arquitetura

ResNet-34 pré-treinada no ImageNet, com a camada final substituída:

```
ResNet-34 backbone (33 camadas convolucionais)  
→ avgpool (512-d feature vector)  
→ Linear(512, 12) ← substituída para 12 classes  
→ Softmax
```

Classes: pawn_w, pawn_b, rook_w, rook_b, knight_w, knight_b, bishop_w, bishop_b, queen_w, queen_b, king_w, king_b

Dataset de treinamento

- **Origem:** células extraídas das imagens do dataset sintético com anotações GT
- **Volume:** ~48 000 imagens de 64×64 px (redimensionadas para 224×224 no dataloader)
- **Split:** 80% treino / 20% validação (estratificado por classe)

- **Desbalanceamento:** peões ~3× mais frequentes que reis/rainhas

Treinamento — Fase 1 (Transfer Learning)

Objetivo: treinar apenas a cabeça classificadora, mantendo features ImageNet intactas.

```
# Congelar backbone
for param in model.parameters():
    param.requires_grad = False
model.fc.requires_grad_(True)

# Hiperparâmetros
optimizer = Adam([fc params], lr=1e-3)
criterion = CrossEntropyLoss()
scheduler = CosineAnnealingLR(optimizer, T_max=10)
epochs    = 10
batch_size = 64
```

Augmentations:

```
transforms.Compose([
    RandomHorizontalFlip(),
    RandomVerticalFlip(),
    RandomRotation(15),
    ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    Resize(224), CenterCrop(224),
    ToTensor(),
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]), #
    ImageNet
])
```

Treinamento — Fase 2 (Fine-tuning)

Objetivo: ajustar as features convolucionais ao domínio específico de peças de xadrez.

```
# Descongelar todo o backbone
for param in model.parameters():
    param.requires_grad = True

# LR discriminativo por grupo de camadas (camadas anteriores recebem LR menor)
param_groups = [
    {"params": model.fc.parameters(), "lr": 3e-5},
    {"params": model.layer4.parameters(), "lr": 3e-5 * 0.3},
    {"params": model.layer3.parameters(), "lr": 3e-5 * 0.09},
    {"params": model.layer2.parameters(), "lr": 3e-5 * 0.027},
    {"params": model.layer1.parameters(), "lr": 3e-5 * 0.0081},
    {"params": model.conv1.parameters(), "lr": 3e-5 * 0.0024},
```

```

]
optimizer = AdamW(param_groups, weight_decay=1e-4)

criterion = CrossEntropyLoss(label_smoothing=0.1)

# Warmup linear (2 épocas) + cosine annealing
scheduler = LambdaLR(optimizer, lr_lambda)

epochs      = 15
grad_clip   = 1.0 # torch.nn.utils.clip_grad_norm_
scaler      = GradScaler() # AMP (Automatic Mixed Precision)

```

Justificativas dos hiperparâmetros:

Hiperparâmetro	Valor	Razão
<code>lr_phase2 = 3e-5</code>	Muito menor que fase 1	Preservar features ImageNet; ajuste fino
<code>lr_decay = 0.3</code> por camada	Exponencialmente menor nas camadas anteriores	Camadas iniciais já têm features genéricas úteis
<code>weight_decay = 1e-4</code>	AdamW regularização	Evita overfitting no fine-tuning
<code>label_smoothing = 0.1</code>	CrossEntropy suavizada	Ajuda classes raras (king, queen)
<code>warmup_epochs = 2</code>	LR cresce linearmente	Evita destruir features no início do fine-tuning
<code>grad_clip = 1.0</code>	Clip do gradiente	Estabiliza treino com LR discriminativo

Resultados

Avaliado em **50 imagens** usando ocupação ground truth (isola o classificador DL da pipeline clássica):

Métrica	Valor
Precisão	91.0%
Recall	91.0%
F1-score	91.0%
TP (tipo+posição corretos)	1 412
Tipo errado (posição certa)	140
FN (peça GT não detectada)	0
FP (predição sem GT)	0

FN = FP = 0 porque a ocupação GT é fornecida diretamente, eliminando erros do classificador clássico.

Acurácia por classe

Classe	Acurácia	Observação
knight_b	96.4%	Silhueta do cavalo é distintiva
pawn_b	96.3%	Classe mais representada
knight_w	95.9%	—
pawn_w	93.8%	—
queen_w	90.8%	Melhorou de 27.7% → 90.8% com fine-tuning
rook_w	90.1%	—
rook_b	89.2%	—
king_w	89.7%	—
bishop_w	87.6%	—
bishop_b	88.2%	—
queen_b	87.8%	—
king_b	84.5%	Classe mais difícil; dataset desbalanceado

Impacto do fine-tuning: antes (1 época de fase 2): F1 = 53.2%; depois (15 épocas com best practices): F1 = **91.0%**. Maior impacto nas classes raras: **king_b** de 14.2% → 84.5%, **queen_w** de 27.7% → 90.8%.

Limitações Conhecidas

Limitação	Impacto	Causa
Material uniforme (madeira/madeira)	F1 clássico ≈ 77%	Baixo contraste peça-fundo
Projeção 3D de peças altas	Falsos positivos em células vizinhas	Torre e rainha "vazam" para células adjacentes
Ambiguidade de 180°	Possível orientação errada	Padrão de xadrez simétrico sob rotação 180°
Limiares fixos	Fraca generalização	Calibrados para dataset sintético
Domain shift CNN	Possível queda de F1 em fotos reais	Treinado apenas em renders Blender

Organização do Código



```
classical-cv/
├─ notebooks/main.ipynb      # Notebook principal com toda a pipeline
├─ src/
│   ├── chess.py             # Funções clássicas de CV
│   ├── piece_classifier.py   # ResNet-34 transfer learning
│   ├── setup.py             # Setup de ambiente e dataset
│   └─ utils.py              # save_fig, save_metrics, create_run_dir
├─ outputs/
│   ├── figures/              # Plots gerados (por execução, com timestamp)
│   ├── results/              # Métricas JSON (por execução)
│   └─ piece_cells/           # Dataset de células para treino da CNN
│       ├── pawn_w/  pawn_b/
│       ├── rook_w/  rook_b/
│       └─ ...
└─ docs/
    ├── intro/                # Apresentação inicial do projeto
    └─ andamento/             # Este documento + slides de andamento
```